CS 336 Assignment #1 (basics): : Building a Transformer LM

Yin Xiaogang

Spring 2025

If you are using LaTeX, you can use `\ifans{}` to type your solutions.
Please tag the questions correctly on Gradescope, otherwise the TAs will take points off if you don't tag questions.

1. Profiling and Benchmarking (8 points)

(a) (4 points) Adam Optimizer
Recall the standard Stochastic Gradient Descent update rule:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)$$

where $t+1$ is the current timestep, $\boldsymbol{\theta}$ is a vector containing all of the model parameters, ($\boldsymbol{\theta}_t$ is the model parameter at time step t, and $\boldsymbol{\theta}_{t+1}$ is the model parameter at time step $t+1$), J is the loss function, $\nabla_{\boldsymbol{\theta}} J_{\text{minibatch}}(\boldsymbol{\theta})$ is the gradient of the loss function with respect to the parameters on a minibatch of data, and α is the learning rate. Adam Optimization[1] uses a more sophisticated update rule with two additional steps.[2]

i. (2 points) First, Adam uses a trick called *momentum* by keeping track of $\mathbf{m}$, a rolling average of the gradients:

$$\begin{aligned}\mathbf{m}_{t+1} &\leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)\\ \boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \alpha \mathbf{m}_{t+1}\end{aligned}$$

where β_1 is a hyperparameter between 0 and 1 (often set to 0.9). **Briefly explain in 2–4 sentences** (you don't need to prove mathematically, just give an intuition) how using $\mathbf{m}$ stops the updates from varying as much and why this low variance may be helpful to learning, overall.

ii. (2 points) Adam extends the idea of *momentum* with the trick of *adaptive learning rates* by keeping track of $\mathbf{v}$, a rolling average of the magnitudes of the gradients:

$$\begin{aligned}\mathbf{m}_{t+1} &\leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)\\ \mathbf{v}_{t+1} &\leftarrow \beta_2 \mathbf{v}_t + (1-\beta_2)(\nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t) \odot \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t))\\ \boldsymbol{\theta}_{t+1} &\leftarrow \boldsymbol{\theta}_t - \alpha \mathbf{m}_{t+1}/\sqrt{\mathbf{v}_{t+1}}\end{aligned}$$

where $\odot$ and / denote elementwise multiplication and division (so $\mathbf{z}\odot\mathbf{z}$ is elementwise squaring) and β_2 is a hyperparameter between 0 and 1 (often set to 0.99). Since Adam divides the update by $\sqrt{\mathbf{v}}$, which of the model parameters will get larger updates? Why might this help with learning? **Briefly explain in 2–4 sentences**.

(b) (4 points) Dropout[3] is a regularization technique. During training, dropout randomly sets units in the hidden layer $\mathbf{h}$ to zero with probability p_{drop} (dropping different units each minibatch), and then multiplies $\mathbf{h}$ by a constant γ. We can write this as:

$$\mathbf{h}_{\text{drop}} = \gamma \mathbf{d} \odot \mathbf{h}$$

[1] Kingma and Ba, 2015, `https://arxiv.org/pdf/1412.6980.pdf`

[2] The actual Adam update uses a few additional tricks that are less important, but we won't worry about them here. If you want to learn more about it, you can take a look at: `http://cs231n.github.io/neural-networks-3/#sgd`

[3] Srivastava et al., 2014, `https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf`

where $\mathbf{d} \in \{0,1\}^{D_h}$ (D_h is the size of $\mathbf{h}$) is a mask vector where each entry is 0 with probability p_{drop} and 1 with probability $(1 - p_{\text{drop}})$. γ is chosen such that the expected value of $\mathbf{h}_{\text{drop}}$ is $\mathbf{h}$:

$$\mathbb{E}_{p_{\text{drop}}}[\mathbf{h}_{\text{drop}}]_i = h_i$$

for all $i \in \{1, \ldots, D_h\}$.

i. (2 points) What must γ equal in terms of p_{drop}? Briefly justify your answer or show your math derivation using the equations given above.

ii. (2 points) Why should dropout be applied during training? Why should dropout **NOT** be applied during evaluation? **Briefly explain in 2–4 sentences**. **Hint:** it may help to look at the dropout paper linked.

2. Distributed Data Parallel Training (42 points)

(a) (5 points) **Problem (distributed_communication_single_node)**

Write a script to benchmark the runtime of the all-reduce operation in the single-node multi-process setup. The example code above may provide a reasonable starting point. Experiment with varying the following settings:

Backend + device type: Gloo + CPU, NCCL + GPU.

all-reduce data size: float32 data tensors ranging over 1MB, 10MB, 100MB, 1GB.

Number of processes: 2, 4, or 6 processes.

Resource requirements: Up to 6 GPUs. Each benchmarking run should take less than 5 minutes.

Deliverable: Plot(s) and/or table(s) comparing the various settings, with 2-3 sentences of commentary about your results and thoughts about how the various factors interact.

Solution: On MacBook pro 2020, use Gloo + CPU, Use

data size	world size	run time
1M	2	0.0033
1M	4	0.0019
1M	6	0.0048
10M	2	0.0112
10M	4	0.0105
10M	6	0.0113
100M	2	0.0795
100M	4	0.1029
100M	6	0.1302
1G	2	0.7987
1G	4	1.0348
1G	6	1.2857

Increasing the number of processes does not necessarily reduce the runtime, especially when dealing with large amounts of data such as 1GB. Adding more processes may even increase the allreduce runtime, possibly due to the significant time consumed by data transmission between processes.

About NCCL + GPU, I have not enough gpus nows.

(b) (5 points) **Problem (naive_ddp)**

Deliverable: Write a script to naively perform distributed data parallel training by all-reducing individual parameter gradients after the backward pass. To verify the correctness of your DDP implementation, use it to train a small toy model on randomly-generated data and verify that its weights match the results from single-process training.

Solution: cs336_systems/naive_ddp.py

(c) (3 points) **Problem (naive_ddp_benchmarking)**

In this nave DDP implementation, parameters are individually all-reduced across ranks after each backward pass. To better understand the overhead of data parallel training, create a script to benchmark your previously-implemented language model when trained with this nave implementation of DDP. Measure the total time per training step and the proportion of time spent on communicating gradients. Collect measurements in the single-node setting (1 node x 2 GPUs) for the XL model size described in §1.1.2.

Deliverable: A description of your benchmarking setup, along with the measured time per training iteration and time spent communicating gradients for each setting.

Solution: cs336_systems/naive_ddp_benchmarking.py. Using Gloo + CPU and small model size, in my Macbook pro 2020, The the measured time per training iteration is 14.94s, the time spent communicating gradients for each setting is 0.6997, about 4.68%

```
3, total time: 14.948427969999988, reduce time: 0.6959458242000209
1, total time: 14.94459973039999, reduce time: 0.7210226140000031
2, total time: 14.9476707444, reduce time: 0.6712903046000065
0, total time: 14.927452500999982, reduce time: 0.7106147133999798
```

(d) (2 points) **Problem (minimal_ddp_flat_benchmarking)** Modify your minimal DDP implementation to communicate a tensor with flattened gradients from all parameters. Compare its performance with the minimal DDP implementation that issues an allreduce for each parameter tensor under the previously-used conditions (1 node x 2 GPUs, XL model size as described in §1.1.2).

Deliverable: The measured time per training iteration and time spent communicating gradients under distributed data parallel training with a single batched all-reduce call. 1-2 sentences comparing the results when batching vs. individually communicating gradients.

Solution: cs336_systems/minimal_ddp_flat_benchmarking.py. Using Gloo + CPU and small model size, in my Macbook pro 2020, The the measured time per training iteration is 23.3792s, the time spent communicating gradients for each setting is 1.51746, about 6.49%. Because small model size and cpu, Using flat increase the overhead.

```
2, total time: 23.38606446599988, reduce time: 1.5045228166001834
1, total time: 23.372208501200113, reduce time: 1.5014608585999667
3, total time: 23.36522756079994, reduce time: 1.580048452200208
0, total time: 23.39337175940018, reduce time: 1.483804502199746
```

(e) (5 points) **Problem (ddp_overlap_individual_parameters)** Implement a Python class to handle distributed data parallel training. The class should wrap an arbitrary PyTorch nn.Module and take care of broadcasting the weights before training (so all ranks have the same initial parameters) and issuing communication calls for gradient averaging. We recommend the following public interface:

```python
def __init__(self, module: torch.nn.Module):
"""
Given an instantiated PyTorch nn.Module to be
parallelized, construct a DDP container that will handle gradient synchronization across ranks.
"""
def forward(self, *inputs, **kwargs):
"""
Calls the wrapped moduleâĂŹs forward() method with the provided positional and keyword arguments.
"""
def finish_gradient_synchronization(self):
"""
When called, wait for asynchronous communication calls to be queued on GPU.
"""
```

To use this class to perform distributed training, well pass it a module to wrap, and then add a call to finish_gradient_synchronization() before we run optimizer.step() to ensure that the optimizer step, an operation that depends on the gradients, may be queued:

```python
model = ToyModel().to(device)
ddp_model = DDP(model)
for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

Deliverable: Implement a container class to handle distributed data parallel training. This class should overlap gradient communication and the computation of the backward pass. To test your DDP class, first implement the adapters [adapters.get_ddp_individual_parameters] and [adapters.ddp_individual_parameters_on_after_backward] (the latter is optional, depending on your implementation you may not need it).

Then, to execute the tests, run uv run pytest tests/test_ddp_individual_parameters.py. We recommend running the tests multiple times (e.g., 5) to ensure that it passes reliably.

Solution: cs336_systems/ddp.py

(f) (1 point) **Problem (ddp_overlap_individual_parameters_benchmarking)**

i. Benchmark the performance of your DDP implementation when overlapping backward pass computation with communication of individual parameter gradients. Compare its performance with our previously-studied settings (the minimal DDP implementation that either issues an all-reduce for each parameter tensor, or a single all-reduce on the concatenation of all parameter tensors) with the same setup: 1 node, 2 GPUs, and the XL model size described in §1.1.2.

Deliverable: The measured time per training iteration when overlapping the backward pass with communication of individual parameter gradients, with 1-2 sentences comparing the results.

Solution: cs336_systems/ddp_overlap_individual_parameters_benchmarking.py. Using Gloo + CPU and small model size, in my Macbook pro 2020, The the measured time per training iteration is 15.4635s, the time spent communicating gradients for each setting is 0.736289 s, about 4.76147 % . The time spent communicating gradients ...

```
3, total time: 15.46724196474861, reduce time: 0.7726576314998965
2, total time: 15.461637291249644, reduce time: 0.7224488822494095
1, total time: 15.46450111025024, reduce time: 0.7475921859995651
0, total time: 15.460556565501065, reduce time: 0.7024589765005658
```

ii. Instrument your benchmarking code (using the 1 node, 2 GPUs, XL model size setup) with the Nsight profiler, comparing between the initial DDP implementation and this DDP implementation that overlaps backward computation and communication. Visually compare the two traces, and provide a profiler screenshot demonstrating that one implementation overlaps compute with communication while the other doesnt.

Deliverable: 2 screenshots (one from the initial DDP implementation, and another from this DDP implementation that overlaps compute with communication) that visually show that communication is or isnt overlapped with the backward pass.

Solution: to be done. I do not have 2 gpus now.

(g) (8 points) **Problem (ddp_overlap_bucketed)** Implement a Python class to handle distributed data parallel training, using gradient bucketing to improve communication efficiency. The class should wrap an arbitrary input PyTorch nn.Module and take care of broadcasting the weights before training (so all ranks have the same initial parameters) and issuing bucketed communication calls for gradient averaging. We recommend the following interface:

```
def __init__(self, module: torch.nn.Module, bucket_size_mb: float):
"""
Given an instantiated
PyTorch nn.Module to be parallelized, construct a DDP container that will handle gradient synchro
across ranks. Gradient synchronization should be bucketed, with each bucket holding
at most bucket_size_mb of parameters.
"""
def forward(self, *inputs, **kwargs):
"""
Calls the wrapped moduleâĂŹs forward() method with the
provided positional and keyword arguments.
```

```
        """
        def finish_gradient_synchronization(self):
        """
        When called, wait for asynchronous communication
        calls to be queued on GPU.
        """
```

Beyond the addition of a bucket_size_mb initialization parameter, this public interface matches the interface of our previous DDP implementation that individually communicated each parameter. We suggest allocating parameters to buckets using the reverse order of model.parameters(), since the gradients will become ready in approximately that order during the backward pass.

Deliverable: Implement a container class to handle distributed data parallel training. This class should overlap gradient communication and the computation of the backward pass. Gradient communication should be bucketed, to reduce the total number of communication calls. To test your implementation, complete [adapters.get_ddp_bucketed], [adapters.ddp_bucketed_on_after_backward], and [adapters.ddp_bucketed_on_train_batch_start] (the latter two are optional, depending on your implementation you may not need them).

Then, to execute the tests, run pytest tests/test_ddp.py. We recommend running the tests multiple times (e.g., 5) to ensure that it passes reliably.

Solution: cs336_systems/ddp.py

(h) (3 points) **Problem (ddp_bucketed_benchmarking)**

i. Benchmark your bucketed DDP implementation using the same config as the previous experiments (1 node, 2 GPUs, XL model size), varying the maximum bucket size (1, 10, 100, 1000 MB). Compare your results to the previous experiments without bucketingdo the results align with your expectations? If they dont align, why not? You may have to use the PyTorch profiler as necessary to better understand how communication calls are ordered and/or executed. What changes in the experimental setup would you expect to yield results that are aligned with your expectations?

Deliverable: Measured time per training iteration for various bucket sizes. 3-4 sentence commentary about the results, your expectations, and potential reasons for any mismatch.

Solution: cs336_systems/ddp_bucketed_benchmarking.py. Using Gloo + CPU and small model size, in my Macbook pro 2020, The the measured time per training iteration is 15.3479s, the time spent communicating gradients for each setting is 0.738696s, about 4.81301% . Because of cpu, no comment.

```
3, total time: 15.350618762498925, reduce time: 0.7534022687495963
2, total time: 15.347716027498791, reduce time: 0.7388711377498112
1, total time: 15.342720182749872, reduce time: 0.7461342754995712
0, total time: 15.350642050000715, reduce time: 0.716377919748993
```

ii. Assume that the time it takes to compute the gradients for a bucket is identical to the time it takes to communicate the gradient buckets. Write an equation that models the communication overhead of DDP (i.e., the amount of additional time spent after the backward pass) as a function 31 of the total size (bytes) of the model parameters (s), the all-reduce algorithm bandwidth (w, computed as the size of each ranks data divided by the time it takes to finish the all-reduce), the overhead (seconds) associated with each communication call (o), and the number of buckets (n_b). From this equation, write an equation for the optimal bucket size that minimizes DDP overhead.

Deliverable: Equation that models DDP overhead, and an equation for the optimal bucket size.

Solution: The total transfer latency across nb buckets is:

$$n_b \cdot \frac{s_b}{w} = nb \cdot \frac{s}{nb \cdot w} = \frac{s}{w}$$

The total fixed overhead across nb calls is:

$$nb \cdot o$$

Thus, the total DDP communication overhead equation is:

$$T(nb) = \frac{s}{w} + nb \cdot o$$

1 Optimal Bucket Size

To derive the optimal bucket size s_b^*, substitute $nb = \frac{s}{s_b}$ into the overhead equation:

$$T(s_b) = \frac{s}{w} + \frac{s \cdot o}{s_b}$$

The derivative of $T(s_b)$ with respect to s_b is negative for all positive s_b, indicating $T(s_b)$ decreases as s_b increases. Constrained by the maximum feasible bucket size $s_{\max}$ (limited by GPU memory), the optimal bucket size is:

$$s_b^* = \min\left(s_{\max}, s\right)$$

Submission Instructions

You shall submit this assignment on GradeScope as two submissions – one for "Assignment 2 [coding]" and another for 'Assignment 2 [written]":

1. Run the `collect_submission.sh` script to produce your `assignment2.zip` file.
2. Upload your `assignment2.zip` file to GradeScope to "Assignment 2 [coding]".
3. Upload your written solutions to GradeScope to "Assignment 2 [written]".